

AI 545 MLOps

Docker Lecture 2: Image Layers, Caching, and Deterministic ML Builds

Lecture Goals

By the end of this lecture, you should be able to:

- Explain Docker image layering.
- Understand Docker build cache mechanics and cache invalidation.
- Structure Dockerfiles for ML projects efficiently.
- Avoid anti-patterns in ML container builds.
- Reason about image determinism in MLOps.

1 Why You Need to Go Deeper

In Lecture 1, you containerized a simple Python script.

That was functional.

Now you move from “it works” to “it is correct, efficient, and reproducible.”

In MLOps, your Docker image becomes:

- The artifact CI (Continuous Integration) builds
- The artifact that serves models
- The artifact that runs training jobs

If the image is unstable, non-deterministic, or inefficient, your entire ML system becomes fragile.

In MLOps, Docker is a first class component.

2 Docker Image Architecture (Deeper Theory)

According to Docker documentation, images are built as a stack of immutable layers.¹

Each instruction in a Dockerfile creates a new read-only layer.

At runtime, Docker uses a Union File System (UnionFS) to combine layers into a single filesystem.²

You should understand this clearly:

- Base image (e.g., python:3.11-slim) → bottom layer
- Your dependency installation → next layer
- Your source code → next layer
- Runtime container layer → writable layer

When a container runs:

¹<https://docs.docker.com/build/concepts/dockerfile/>

²<https://docs.docker.com/storage/storagedriver/>

- All lower layers are read-only.
- The container gets a thin writable layer on top.

When the container stops and is removed, that writable layer disappears.

This is why containers are ephemeral.

MLOps Insight

If you write logs or artifacts inside the container filesystem without mounting volumes, they disappear when the container is removed.

You will address volumes later.

3 Build Cache and Cache Invalidation

Docker uses a build cache.³

Docker compares:

- The Dockerfile instruction
- The content of files used in that instruction

If nothing changes, Docker reuses the cached layer.

If something changes, all subsequent layers rebuild.

This is critical.

Cache Invalidation Rule

If layer N changes, layers N+1, N+2, ..., must rebuild.⁴

In ML projects:

- Code changes frequently
- Dependencies change rarely

Therefore, dependency installation must appear before source code copy.

4 Determinism in ML Images

Now you think like an MLOps engineer.

If your Dockerfile says:

```
FROM python:3.11-slim
RUN pip install scikit-learn
```

This is NOT deterministic.

Why?

Because:

- pip installs the latest version by default
- Tomorrow, a new version may be released

Now your image changes without you changing code.

That breaks reproducibility.

³<https://docs.docker.com/build/cache/>

⁴<https://docs.docker.com/build/cache/invalidation/>

Correct Approach

You pin versions (BTW where and how do you get these exact versions remember?):

```
scikit-learn==1.4.2
mlflow==2.9.1
```

And ideally, you use:

```
pip install --no-cache-dir -r requirements.txt
```

In MLOps, deterministic builds are mandatory.

5 Now You Perform Controlled Experiments

Create a new project folder:

```
docker-demo-2/
```

have requirements.txt file and in it have the following:

```
scikit-learn==1.4.2
mlflow==2.9.1
```

6 Step 1: Create a Poorly Structured Dockerfile

Write this:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "train.py"]
```

Now build:

```
docker build -t mlops-bad:0.1 .
```

Now modify only train.py.

Now rebuild.

```
docker build -t mlops-bad:0.2 .
```

Observe carefully between the two tags:

- Dependency installation reruns.
- Build time increases.

You just invalidated cache unnecessarily.

7 Step 2: Write a Production-Style ML Dockerfile

Now rewrite it properly:

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
# Install system-level dependencies first (if needed)
```

```

RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Copy only dependency file
COPY requirements.txt .

# Install Python dependencies
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copy source code
COPY . .

CMD ["python", "train.py"]

```

Now build:

```
docker build -t mlops-good:0.1 .
```

Now change only source code, maybe a new print statement, whatever.

Now rebuild.

```
docker build -t mlops-good:0.2 .
```

Observe:

- Dependency layer reused.
- Build faster.

This is correct MLOps layering.

8 Introducing .dockerignore

Now create:

```
.dockerignore
```

Add:

```

.git
__pycache__
*.pyc
mlruns/
data/
.dvc

```

According to Docker documentation, the build context includes everything in the directory unless excluded.⁵

If you forget this file:

- Your dataset may be sent to Docker daemon.
- Build becomes slow.
- Image becomes huge.

In MLOps, images should contain:

- Code
- Runtime dependencies

⁵<https://docs.docker.com/build/concepts/context/>

They should NOT contain:

- Raw datasets
- Experiment artifacts
- Git history

9 Inspect Image Internals

Now run:

```
docker history mlops-good:0.1
```

You will see each layer and its size.

Now run:

```
docker image inspect mlops-good:0.1
```

Observe:

- Metadata
- Environment
- Entrypoint

This is what CI/CD systems will later use.

10 In-Class Activity

Now you perform a controlled optimization exercise.

Task

You are given:

```
requirements.txt  
train.py  
Dockerfile (naive)
```

You must:

- Reorder the Dockerfile to minimize rebuild time.
- Add version pinning in requirements.
- Add a proper .dockerignore.
- Build the image twice:
 - First after dependency change
 - Second after only code change
- Compare build times.

11 Conceptual Wrap-Up

You now understand:

- Docker images are layered and immutable.
- Cache invalidation follows strict rules.
- Instruction ordering affects rebuild efficiency.
- Deterministic builds require dependency pinning.

- Images should respect MLOps artifact boundaries.

You now have:

- Code versioned by Git
- Data versioned by DVC
- Models versioned by MLflow
- Environment versioned and built deterministically by Docker

In the next lecture, you will containerize an MLflow-backed inference API and transform models into services.